Graph Neural Networks

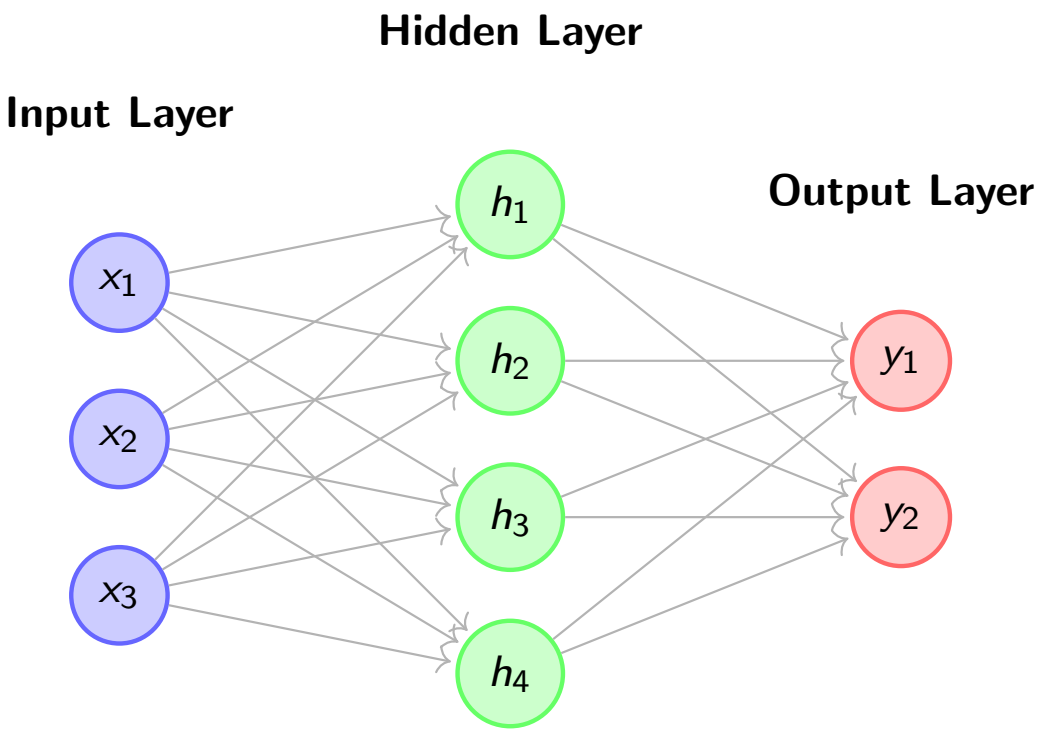
George Michailidis

gmichail@ucla.edu

STAT 426

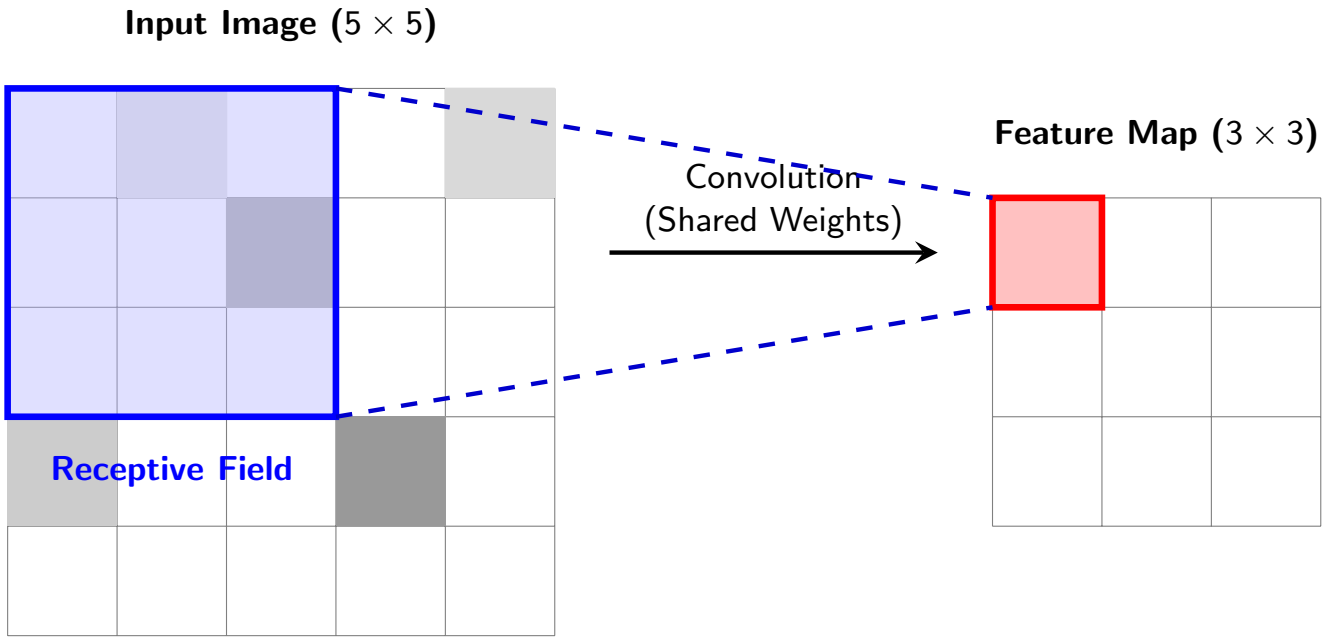
Recap: Tabular Data

- When dealing with standard classification (categorical outcome) or regression tasks and tabular data, we traditionally rely on Multi-Layer Perceptrons (MLPs).
- MLPs utilize a straightforward architecture consisting of an input layer, hidden layers, and an output layer to map the predictors/features to the outcome variable(s).



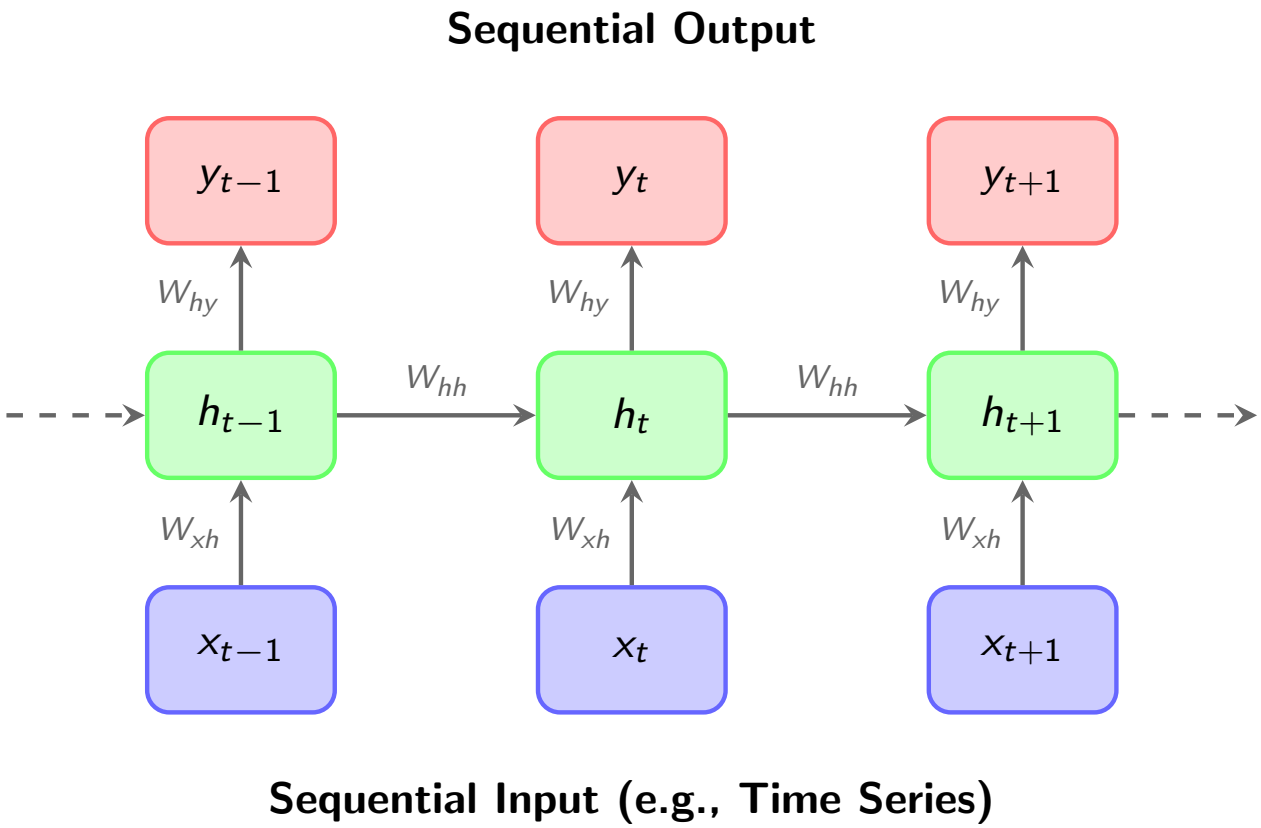
Recap: Image Data

- Another common form of grid-structured data is images.
- Images are arranged in neat, uniform 2D/3D grids of pixels.
- This topology is perfectly suited for Convolutional Neural Networks (CNNs) which rely on fixed-size sliding windows.



Recap: Sequential Data

- Time evolving (or more generally ordered) observations.
- We typically process this sequential data using Recurrent Neural Networks (RNNs) and their variants (LSTMs/GRUs).



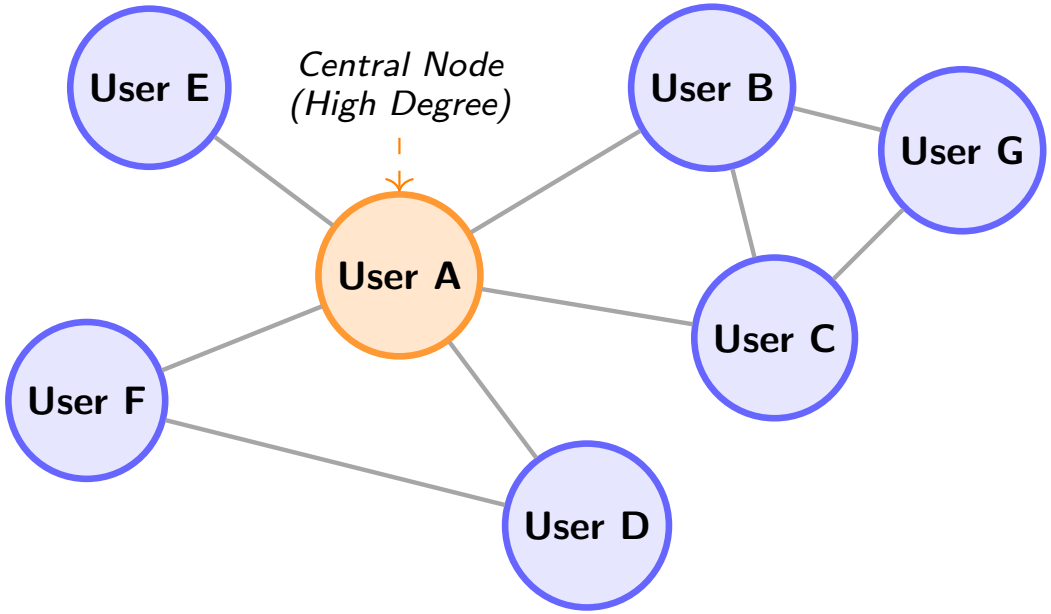
Graph Data

Graphs are vital because they accurately model real-world phenomena across various domains; examples include:

- Social networks: Modeling relationships and tracking influence propagation.
- Molecules: Predicting chemical properties that depend heavily on structural layout.
- Citation networks: Forming the basis of knowledge graphs and recommendation systems.
- Traffic systems: Routing and predicting congestion dynamically.

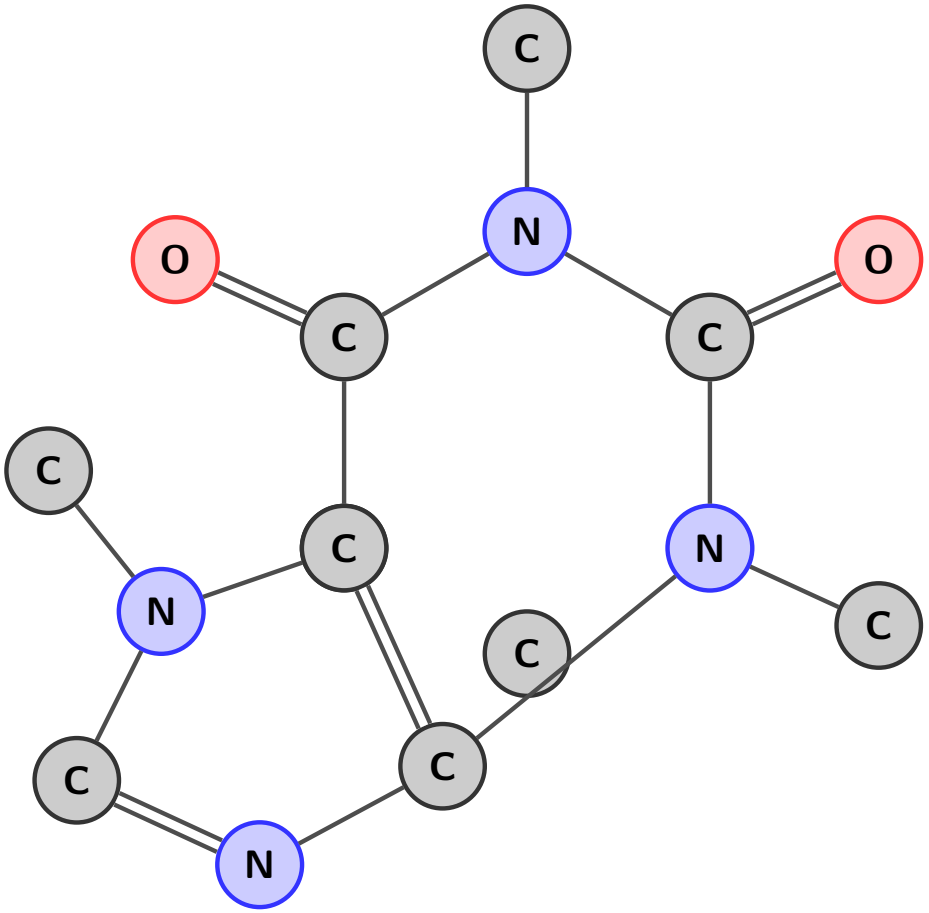
Translating to Graph Representations: Social Networks

- To process complex structures computationally, we translate them into mathematical graphs.
- In a Social Network, individuals or accounts become nodes, and their interactions (e.g., follows, friendships, messages) become edges.



Translating to Graph Representations: Molecules

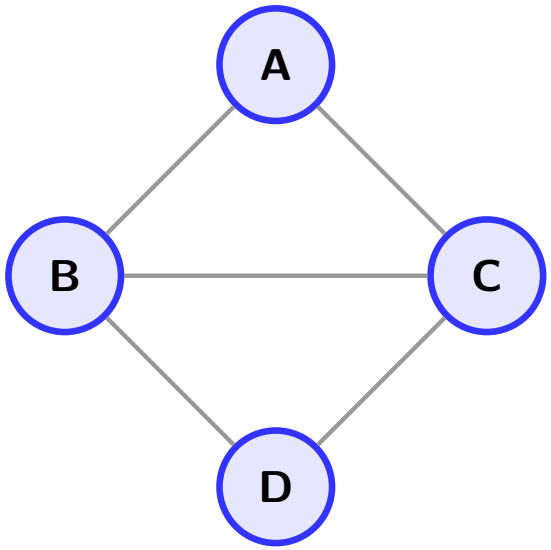
- Similarly, physical structures like molecules map perfectly to graphs.
- In this topological graph of a caffeine molecule, atoms become nodes (with elemental features) and chemical bonds become edges.



Anatomy of a Graph: Topology

A graph $G = (V, E)$ consists of:

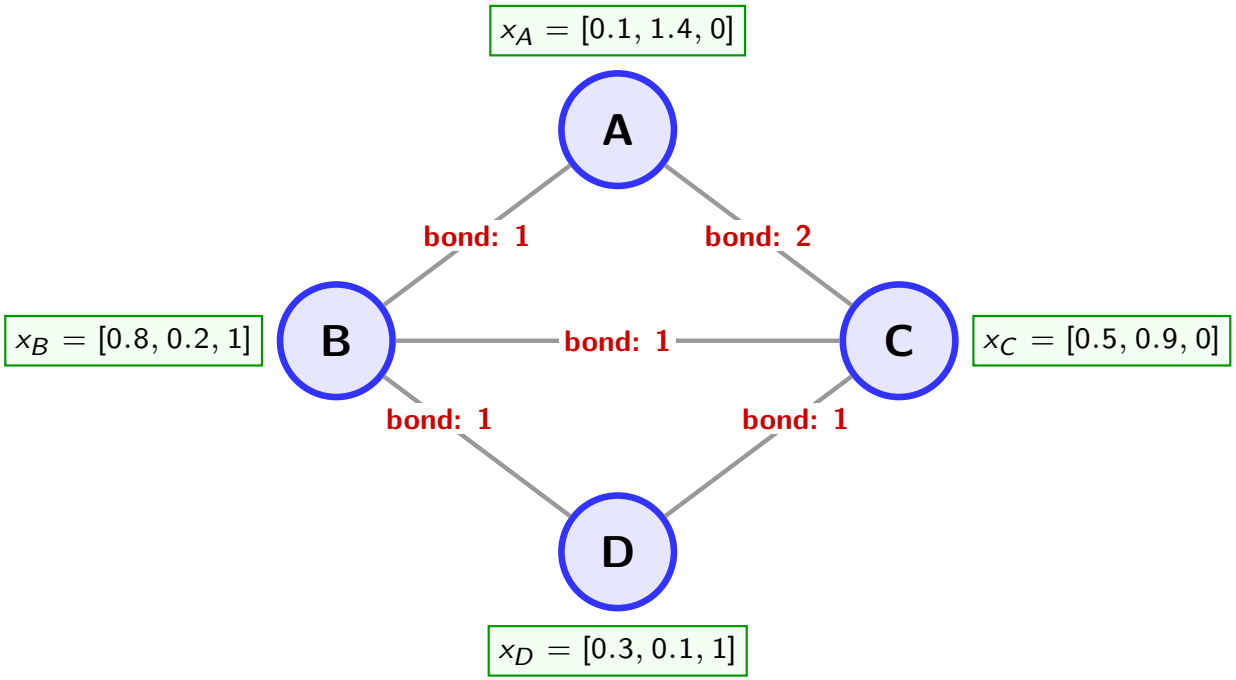
- Nodes/Vertices (V): Represent the core entities (e.g., the individual atoms in our caffeine molecule, or users in a social network).
- Edges (E): Represent the direct connections or relationships between those entities (e.g., chemical bonds, friendships).



Anatomy of a Graph: Features (Attributes/Covariates)

A graph $G = (V, E, X, E_f)$ with attributes adds features to the structure:

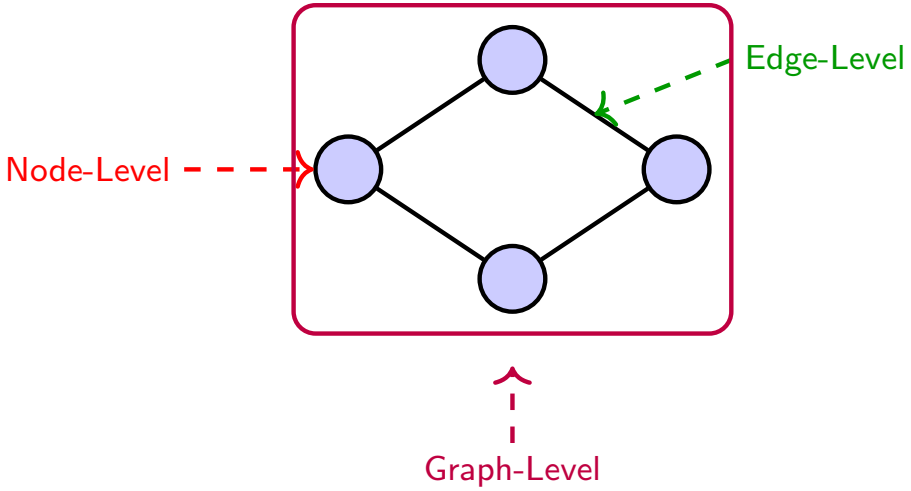
- Node Features ($X \in \mathbb{R}^{|V| \times D}$): A vector describing the node. For a molecule, this could be atomic number, charge, and mass. For a user, it could be age, location, and post frequency.
- Edge Features (E_f): Vectors or scalars describing the relationship, such as the type of chemical bond (single/double) or the strength of a friendship.



Prediction Tasks on Graphs

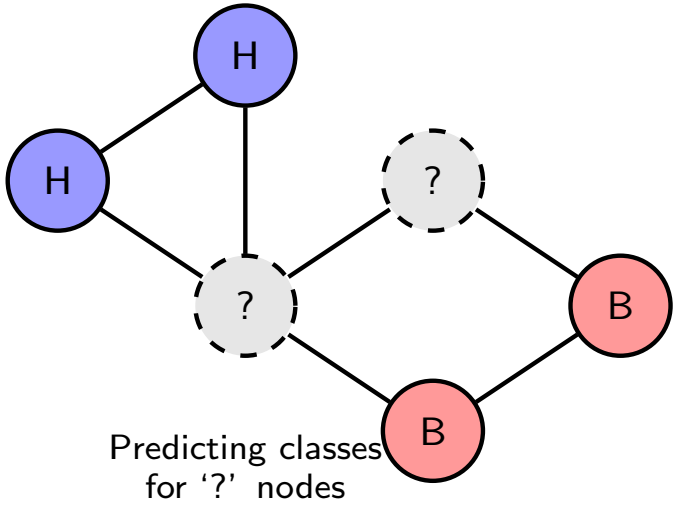
Just as we have image classification, semantic segmentation, and object detection for CNNs, graphs have three primary levels of prediction tasks.

1. Node-level: Predicting properties of individual nodes.
2. Edge-level: Predicting properties of pairs of nodes (links).
3. Graph-level: Predicting properties of the entire graph.



1. Node-Level Tasks (Node Classification)

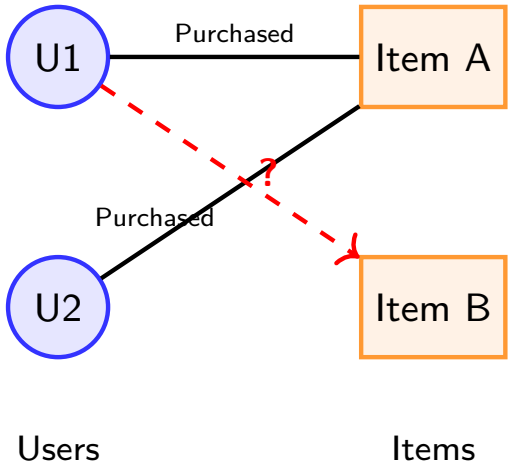
- Goal: Predict the label or feature of an unlabeled node.
- Analogy: Semantic segmentation (pixel-level prediction).
- Examples:
  - Spam/bot detection in social networks.
  - Document classification in citation networks (e.g., Cora dataset).



Key Insight: We leverage homophily—connected nodes tend to share similar properties.

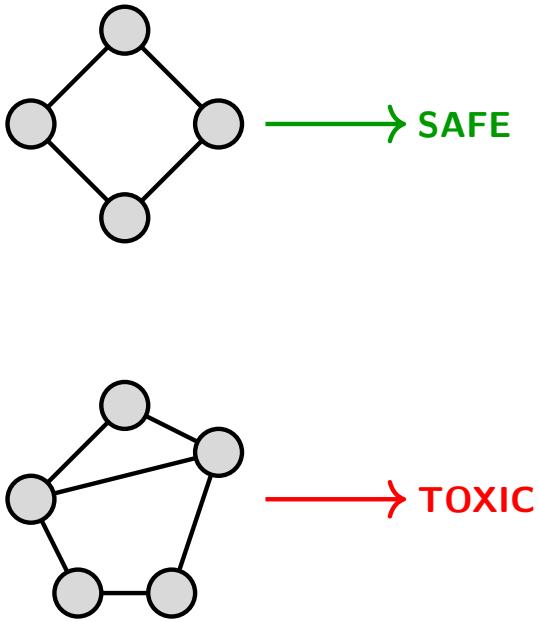
2. Edge-Level Tasks (Link Prediction)

- Goal: Predict whether an edge exists between two nodes, or the type of the edge.
- Examples:
  - Recommendations: Will User A buy Item B? (Bipartite graph).
  - Social Networks: Friend recommendations.
  - Knowledge Graphs: Completing facts (e.g., [Rick Schoenberg] $\xrightarrow{\text{educated_at}}$ [?]).



3. Graph-Level Tasks (Graph Classification)

- Goal: Predict a property of the entire graph.
- Analogy: Image classification.
- Examples:
  - Drug Discovery: Is this molecule toxic? Will it bind to a specific protein?
  - Program Analysis: Does this control-flow graph contain a vulnerability?

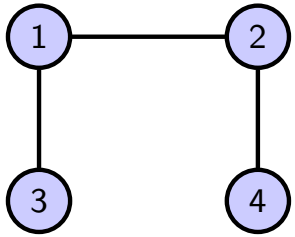


More on Graph with Attributes Representations

As mentioned, a graph $G = (V, E)$ consists of vertices (nodes) V and edges E.

To feed this into a neural network architecture, we need matrices:

- **Adjacency Matrix ($A \in \mathbb{R}^{N \times N}$):** Captures topology. $A_{ij} = 1$ if edge exists between i and j.
- **Node Feature Matrix ($X \in \mathbb{R}^{N \times D}$):** Captures attributes. Row i contains the D-dim feature for node i.
- **Edge Matrix:** (Optional) Attributes like distance/type/strength.



$$A = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$

$$X = \begin{bmatrix} x_{11} & x_{12} \\ x_{21} & x_{22} \\ x_{31} & x_{32} \\ x_{41} & x_{42} \end{bmatrix}$$

Why not just use an MLP?

The Naive Approach: Flatten the adjacency matrix $\mathbf{A} \in \mathbb{R}^{N \times N}$ and feature matrix $\mathbf{X} \in \mathbb{R}^{N \times F}$, concatenate them, and feed the giant vector into a standard Multi-Layer Perceptron (MLP).

Why this can fail catastrophically:

1. Size Variance: MLPs require fixed-size input layers. Real-world graphs have wildly varying numbers of nodes N.
2. Extreme Sparsity: $\mathbf{A}$ is mostly empty. Real networks have $O(N)$ edges, but the matrix requires $O(N^2)$ space. An MLP would waste millions of parameters multiplying by zeros.
3. Permutation Sensitivity: The ordering of nodes in an adjacency matrix is completely arbitrary!

The Permutation Problem

If we swap the arbitrary indices of two nodes, the underlying graph is physically isomorphic, but the adjacency matrix $\mathbf{A}$ changes completely.

$$\mathbf{A}_{orig} = \begin{bmatrix} 0 & \mathbf{1} & 0 \\ \mathbf{1} & 0 & \mathbf{1} \\ 0 & \mathbf{1} & 0 \end{bmatrix} \qquad \mathbf{A}_{new} = \begin{bmatrix} 0 & \mathbf{1} & \mathbf{1} \\ \mathbf{1} & 0 & 0 \\ \mathbf{1} & 0 & 0 \end{bmatrix}$$

An MLP treats $\mathbf{A}_{orig}$ and $\mathbf{A}_{new}$ as entirely different inputs. We require architectures that are inherently permutation equivariant (node tasks) or permutation invariant (graph tasks).

Remark: Note the analogy with invariance and equivariance notions leveraged in CNNs for image data.

Why not just use a CNN?

CNNs are very good at extracting local features using shared weights (filters). Why not apply a filter over the graph?

Euclidean Data (Images)

- Fixed topological grid.
- A 3×3 filter *always* processes exactly 9 pixels.
- Strict spatial ordering (up, down, left, right).

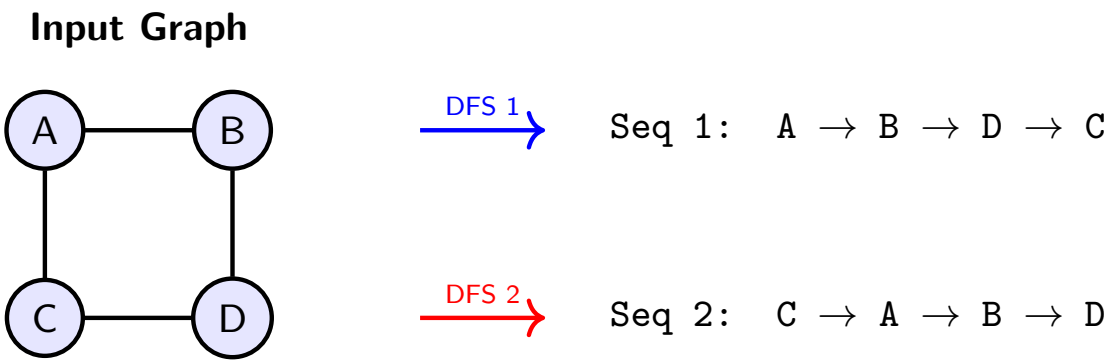
Non-Euclidean Data (Graphs)

- Arbitrary topology.
- Node A might have 2 neighbors; Node B might have 200; Node C might have 0.
- No concept of spatial direction or ordering.

The Core Challenge: We need a convolution operation that dynamically adapts to a local neighborhood of any size and any ordering.

Why not just use an RNN?

RNNs handle arbitrary lengths well. What if we just "traverse" the graph (e.g., via Depth First Search algorithm or random walk) and feed the resulting sequence of nodes into an RNN?



Why this fails:

- Loss of structural integrity: A graph has no natural "start" or "end".
- Sequence Bias: The RNN will treat `Seq 1` and `Seq 2` as drastically different inputs, producing completely different embeddings for the exact same underlying graph structure.

What about Attention Mechanisms?

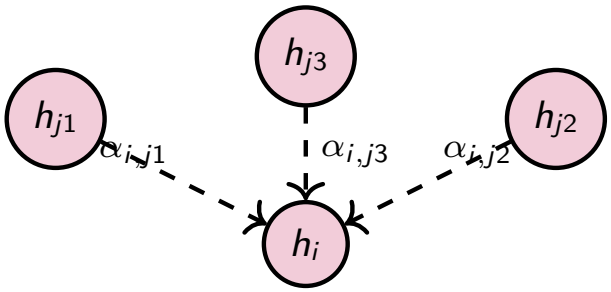
If MLPs, CNNs, and RNNs fail, what about Transformers and Attention?
Actually, attention is the perfect fit for graphs!

Self-Attention (Transformers)

- Computes pairwise importance between *all* items in a set.
- Inherently permutation equivariant (order doesn't matter unless positional encoding is added!).
- Treats data as a fully-connected graph.

Graph Attention (GAT)

- We can apply the exact same attention mechanism, but mask it using the Adjacency Matrix **A**.
- Nodes only attend to their immediate topological neighbors.
- Solves varying neighborhood sizes elegantly!



Target node i aggregates features from neighbors j weighted by learnable attention scores α.

How do we build a Graph Neural Network?

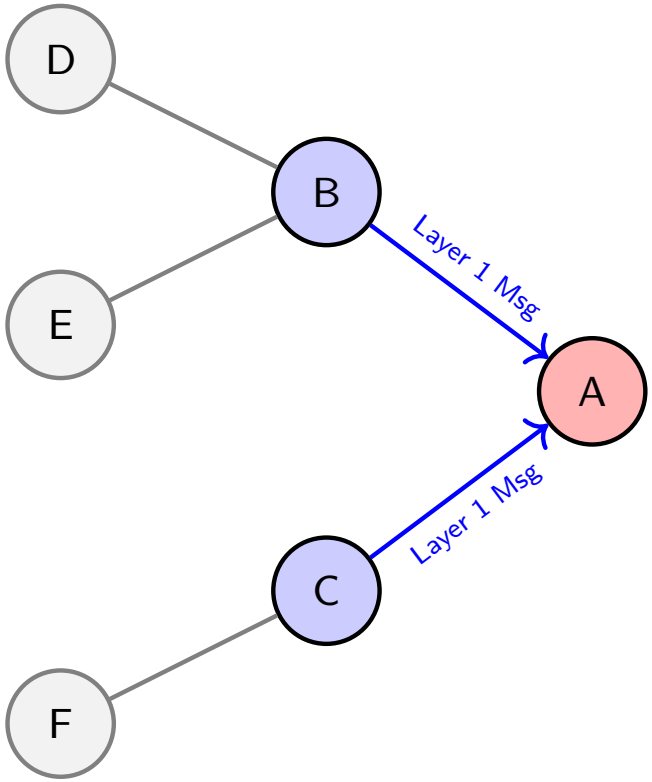
- We have our graph topology $G = (V, E)$ and node features X.
- Our goal: Compute a new, context-aware embedding h_v for every node v that captures both its own features and the graph structure around it.
  - *Note: Advanced graphs also have edge features (e.g., chemical bond types), which we will incorporate later!*
- How do we compute this? By having nodes "talk" to each other!
- This brings us to the fundamental algorithm of almost all modern GNNs: Message Passing.

The Core Idea: Neighborhood Aggregation

- To compute the representation of a target node, we aggregate information from its immediate (1-hop) neighbors.
- Just like in a CNN, where stacking 3×3 filters expands the receptive field, stacking K GNN layers expands the graph receptive field to K hops.

Expanding the Receptive Field: Layer 1 (1-Hop)

Let's look at Target Node A. In the first layer, every node aggregates from its direct neighbors.

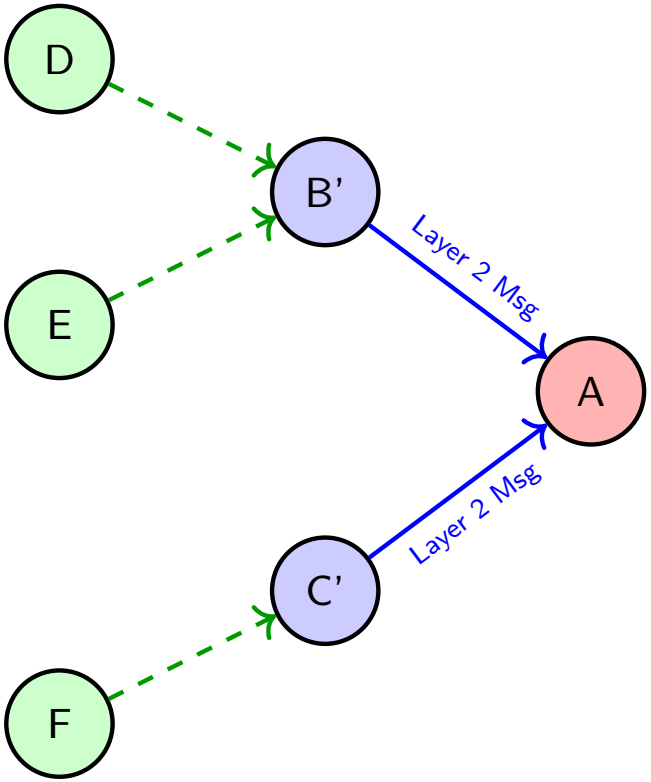


After Layer 1:
$h_A^{(1)}$ contains info from {A, B, C}.

Crucially, at this same step, B aggregated from D and E!

Expanding the Receptive Field: Layer 2 (2-Hop)

In the **second layer**, Node A aggregates from B and C again. But B and C now contain information from *their* neighbors!



After Layer 2:
Because B' contains {B, D, E}, when it passes a message to A, **A indirectly learns about D and E**.

$h_A^{(2)}$ now contains info from its **2-hop neighborhood**: {A, B, C, D, E, F}.

The Message Passing Algorithm (Gilmer et al., 2017)

Every layer t of a Message Passing Neural Network (MPNN) consists of three distinct steps for each node i:

1. Message: Generate a message from each neighbor j to target i.

$$m_{j \to i}^{(t)} = M_t \left(h_i^{(t-1)}, h_j^{(t-1)}, e_{ji} \right)$$

2. Aggregate: Combine all incoming messages from the neighborhood $\mathcal{N}(i)$ into a single vector. *(Must be permutation invariant!)*

$$m_i^{(t)} = \text{AGGREGATE} \left(\{ m_{j \to i}^{(t)} \mid j \in \mathcal{N}(i) \} \right)$$

3. Update: Update the target node's embedding using its old state and the aggregated message.

$$h_i^{(t)} = U_t \left(h_i^{(t-1)}, m_i^{(t)} \right)$$

Deconstructing the Message: What exactly is h_j? - I

When neighbor j sends $h_j^{(t-1)}$ to our target node i, what information is actually moving across the edge?

- Base Case ($t = 1$): At the very first layer, $h_j^{(0)}$ is simply X_j. It is the raw, original feature vector of node j (e.g., a word embedding, a user's age and income, or atomic properties). It contains no graph topology.
- Recursive Case ($t > 1$): By layer 2, $h_j^{(1)}$ has already gone through its own message passing. Therefore, $h_j^{(1)}$ is a mathematical mixture of:
  1. Node j's original features.
  2. The features of all nodes connected to j.
- The Takeaway: When j passes $h_j^{(t-1)}$ to i, it is not just sending its own features. It is sending a highly compressed summary of its entire $(t-1)$-hop neighborhood topology.

Deconstructing the Update: Merging Self and Context

Step 3 is the Update step: $h_i^{(t)} = U_t\left(h_i^{(t-1)}, m_i^{(t)}\right)$.

Why do we explicitly need $h_i^{(t-1)}$ here?

- The Danger of Forgetting: If U_t only looked at the aggregated neighbor messages ($m_i^{(t)}$), node i would instantly forget its own unique features. It would just become the average of its friends!

Deconstructing the Update: Merging Self and Context

Step 3 is the Update step: $h_i^{(t)} = U_t\left(h_i^{(t-1)}, m_i^{(t)}\right)$.

Why do we explicitly need $h_i^{(t-1)}$ here?

- The Danger of Forgetting: If U_t only looked at the aggregated neighbor messages ($m_i^{(t)}$), node i would instantly forget its own unique features. It would just become the average of its friends!
- The Mechanics: U_t must be a learnable function that balances *"who I am"* with *"who my friends are"*.

An Example of an Update Function

Typically, U_t is implemented as a concatenation followed by a linear layer and a non-linear activation (like ReLU):

$$h_i^{(t)} = \text{ReLU}\left(W^{(t)} \cdot \left[h_i^{(t-1)} \| m_i^{(t)}\right]\right)$$

- $\left[\cdot \| \cdot\right]$: Concatenates the node's history with the new neighborhood message, explicitly preserving the node's distinct identity.
- $W^{(t)}$: A learnable weight matrix that decides how much to trust the node's own history versus the neighborhood's context.

What about e_{ji}?

In our first slide, the message generation formula included a third term:

$$m_{j \to i}^{(t)} = M_t \left(h_i^{(t-1)}, h_j^{(t-1)}, e_{ji} \right)$$

Why do we need Edge Features (e_{ji})?

- In many real-world domains, the type of relationship is just as important as the nodes themselves.
- Chemistry: Nodes are atoms (Carbon, Oxygen), but edges are bonds. A single bond passes electrons (and information) very differently than a double or aromatic bond.
- Knowledge Graphs: Node A (Paris) and Node B (France) are connected, but the edge feature tells us *how* (e.g., "is_capital_of" vs. "is_located_in").
- Social Networks: An edge might contain the frequency of interaction, or a binary state (Friend vs. Blocked).

Upgrading the Message Function with Edges

How do we actually inject e_{ji} into the math? Standard node-only models just multiply neighbor features by a weight matrix. With edge features, the message function M_t becomes slightly more complex.

Common Strategy 1: Concatenation

- We simply glue the edge feature to the node features before passing them through a neural network layer (MLP):

$$m_{j\to i}^{(t)} = \text{MLP}\left(\left[h_i^{(t-1)} \| h_j^{(t-1)} \| e_{ji}\right]\right)$$

- The network learns to naturally weight the neighbor's information based on the context of the connection.

Common Strategy 2: Edge-Conditioned Weights

- We use the edge feature to dynamically generate the weight matrix itself (e.g., Edge-Conditioned Convolution - ECC):

$$m_{j\to i}^{(t)} = \text{MLP}_{edge}(e_{ji}) \cdot h_j^{(t-1)}$$

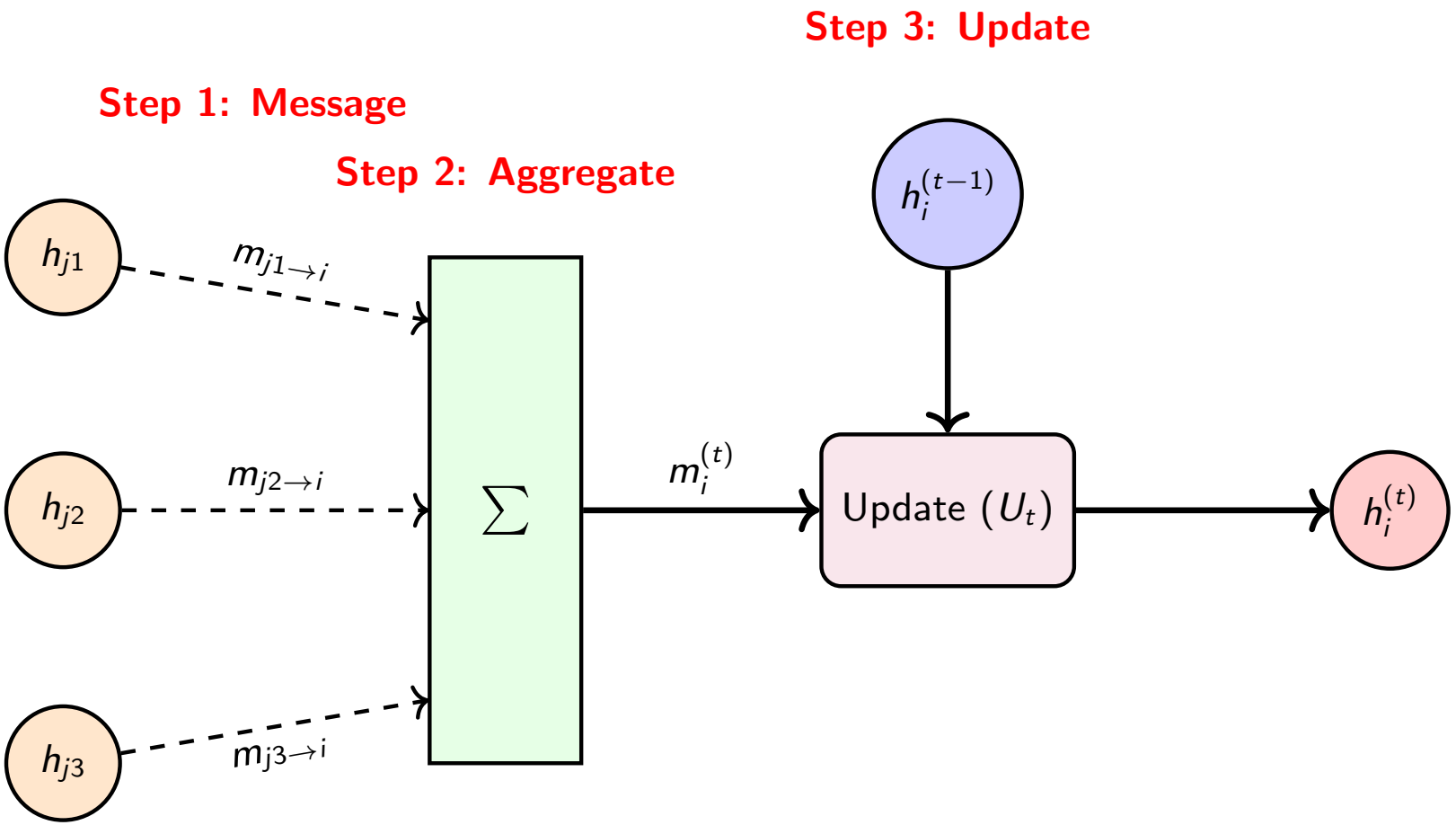
Advanced MPNNs: Do Edge Features Update?

- Standard GNNs: Edge features e_{ji} are treated as static attributes. They influence how node messages are created, but the edges themselves never change across layers.
- Advanced GNNs (e.g., Interaction Networks): We can elevate edges to be "first-class citizens." If nodes can update their state by looking at edges, edges should be able to update their state by looking at nodes!
- In this paradigm, we add an Edge Update Step alongside the Node Update step:

$$e_{ji}^{(t)} = U_{edge}\left(e_{ji}^{(t-1)}, h_i^{(t-1)}, h_j^{(t-1)}\right)$$

- *Result:* As the network gets deeper, a "single bond" embedding might mathematically evolve into a "highly stressed single bond" based on the shifting states of the two atoms it connects.

Visualizing the 3 Steps



Note: All standard GNN architectures are just specific mathematical choices for M_t, AGGREGATE, and U_t.

Architecture 1: Graph Convolutional Networks (GCN)

GCN (Kipf & Welling, 2017) is the most famous baseline. It uses a fast, first-order approximation of spectral graph convolutions.

GCN Propagation Rule

$$H^{(t)} = \sigma\left(\widehat{D}^{-1/2}\widehat{A}\,\widehat{D}^{-1/2}H^{(t-1)}W^{(t-1)}\right)$$

Let's look at this from the Message Passing perspective for a single node i:

$$h_i^{(t)} = \sigma\left(\sum_{j\in\mathcal{N}(i)\cup\{i\}} \frac{1}{\sqrt{\deg(i)\deg(j)}} h_j^{(t-1)} W^{(t-1)}\right)$$

The GCN Propagation Rule

The Equation:

$$H^{(t)} = \sigma\left(\widehat{D}^{-1/2}\widehat{A}\,\widehat{D}^{-1/2}H^{(t-1)}W^{(t-1)}\right)$$

Matrices Involved:

- $\widehat{A} = A + I$: Adjacency matrix with self-loops (node keeps its own features).
- $\widehat{D}$: Degree matrix of $\widehat{A}$ (counts neighbors + self).
- $\widehat{D}^{-1/2}\widehat{A}\,\widehat{D}^{-1/2}$: Symmetrically normalized adjacency matrix.

Symmetric Normalization:

- **Stability:** Prevents exploding/vanishing features during aggregation.
- **Sender Discount:** Down-weights messages from nodes that broadcast to everyone.
- **Receiver Discount:** Down-weights messages for nodes that listen to everyone.

More on the GCN Architecture

$$h_i^{(t)} = \sigma\left(\sum_{j \in \mathcal{N}(i) \cup \{i\}} \frac{1}{\sqrt{\deg(i)\deg(j)}} h_j^{(t-1)} W^{(t-1)}\right)$$

- Message: $h_j^{(t-1)} W^{(t-1)}$ (Multiply neighbor features by a weight matrix).
- Aggregate: Summation, scaled by the degrees of both sender and receiver $\left(\frac{1}{\sqrt{\deg(i)\deg(j)}}\right)$.
- Update: Apply a non-linearity σ (like ReLU). Note that GCN adds self-loops ($\widehat{A} = A + I$), meaning a node passes a message to itself!

More on the GCN Architecture

$$h_i^{(t)} = \sigma\left(\sum_{j \in \mathcal{N}(i) \cup \{i\}} \frac{1}{\sqrt{\deg(i)\deg(j)}} h_j^{(t-1)} W^{(t-1)}\right)$$

- Message: $h_j^{(t-1)} W^{(t-1)}$ (Multiply neighbor features by a weight matrix).
- Aggregate: Summation, scaled by the degrees of both sender and receiver $\left(\frac{1}{\sqrt{\deg(i)\deg(j)}}\right)$.
- Update: Apply a non-linearity σ (like ReLU). Note that GCN adds self-loops ($\widehat{A} = A + I$), meaning a node passes a message to itself!

Pros: Simple, highly efficient ($O(|E|)$ scaling), great baseline.
Cons: Treats all neighbors equally (based only on degree structure), suffers heavily from over-smoothing if deep.

The Scalability Problem of GCN

- Transductive Learning: Standard GCN requires the *entire* graph adjacency matrix $\widehat{A}$ to be in memory simultaneously.
- What if the graph has 2 billion nodes (e.g., Facebook)? We cannot fit $\widehat{A}$ in GPU memory!
- What if a completely new node joins the network tomorrow? GCN would require recomputing the whole graph.

Digression: Spectral vs. Spatial GNNs

Historically, graph convolutions developed from two very different mathematical viewpoints. It is crucial to understand the distinction:

1. The Spectral Perspective
   - Origin: Graph Signal Processing.
   - Mechanism: Uses the Graph Laplacian and its eigenvectors to translate signals into the frequency domain (the Graph Fourier Transform), applies a filter, and transforms back.
   - Limitation: Computationally expensive ($\mathcal{O}(N^3)$ eigen-decomposition) and strictly tied to a fixed graph structure (transductive).
2. The Spatial Perspective
   - Origin: Local routing / Message Passing.
   - Mechanism: Operates directly on the physical graph topology by aggregating features from immediate neighbors.
   - Advantage: Conceptually intuitive, highly scalable (via mini-batching), and easily handles dynamic or entirely new graphs (inductive learning).

The 2017 GCN paper actually started in the *Spectral* domain. It established that a first-order approximation of a spectral filter is mathematically equivalent to a simple *Spatial* message-passing step! Today, almost all modern GNNs operate spatially.

Architecture 2: GraphSAGE

GraphSAGE (Hamilton et al., 2017) solves this by introducing an Inductive framework with Neighborhood Sampling.

- Instead of using *all* neighbors, GraphSAGE randomly samples a fixed number of neighbors (e.g., 10-25) for each node.
- This breaks the dependency on the full graph, allowing standard mini-batch training!
- Because it learns an aggregation *function* (rather than relying on a static matrix $\widehat{A}$), it can generate embeddings for entirely unseen nodes.

More on the GraphSAGE Architecture

GraphSAGE Update Rule

$$m_{\mathcal{N}(i)}^{(t)} = \text{AGGREGATE}\left(\left\{h_j^{(t-1)} \mid j \in \text{Sample}(\mathcal{N}(i))\right\}\right)$$

$$h_i^{(t)} = \sigma\left(W^{(t)} \cdot \text{CONCAT}\left(h_i^{(t-1)}, m_{\mathcal{N}(i)}^{(t)}\right)\right)$$

- Aggregate: GraphSAGE proposes several aggregator choices:
  - **Mean:** Element-wise mean of neighbors.
  - **Max-Pooling:** Apply an MLP to neighbors, then element-wise max.
  - **LSTM:** Apply an LSTM to randomly permuted neighbors.
- Update: Notice it concatenates the old state h_i with the message $m_{\mathcal{N}(i)}$, rather than just adding them like GCN. This helps preserve central node information.

The Weighting Problem of GCN & GraphSAGE

- GCN scales messages based strictly on node degree.
- GraphSAGE (with Mean aggregation) weights all sampled neighbors equally.
- Key issue:In real life networks, some neighbors are much more important than others! (e.g., a close family member vs. a distant acquaintance on a social network).

Architecture 3: Graph Attention Networks (GAT)

GAT (Veličković et al., 2018) introduces Attention to graph edges.

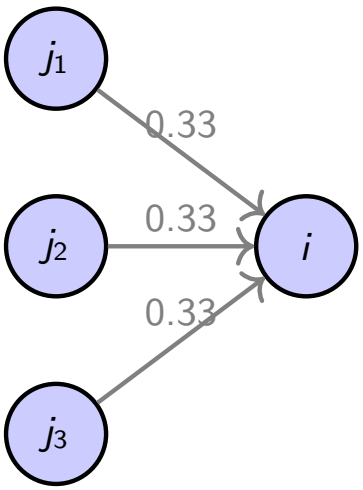
Instead of fixed weights, the network *learns* an attention coefficient α_{ij} indicating how important neighbor j's features are to node i.

GAT Layer

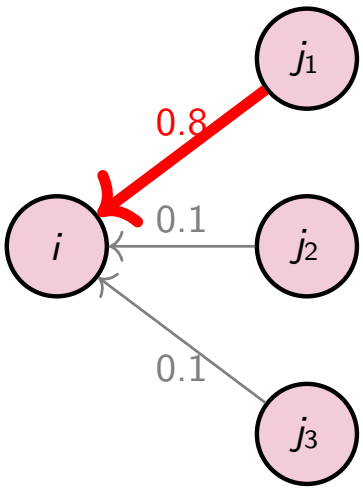
$$h_i^{(t)} = \sigma\left(\sum_{j \in \mathcal{N}(i) \cup \{i\}} \alpha_{ij} W h_j^{(t-1)}\right)$$

Visualizing GAT vs GCN

GCN (Fixed Structural Weights)



GAT (Learned Attention Weights)



Note: GAT is effectively a Transformer that uses the graph adjacency matrix as a strict attention mask!

Architecture 3': Beyond Local Neighborhoods - Graph Transformers

While GAT applies attention to local edges, Graph Transformers (GTs) treat the entire graph as a fully connected sequence of nodes.

Full Node-to-Node Attention

- The Concept: Every node computes an attention score with *every* other node in the graph, regardless of whether a physical edge exists between them.
- Addressing Limitations: Standard Message Passing (and GAT) suffers from over-squashing because distant signals must squeeze through local neighborhood bottlenecks. GTs solve this by creating direct $\mathcal{O}(1)$ paths between all nodes.
- The Trade-off: Computing dense attention over all N nodes requires $\mathcal{O}(N^2)$ compute and memory, meaning standard GTs are primarily used on smaller graphs (like molecules).

Scaling Graph Transformers

Full node-to-node attention in standard Graph Transformers scales quadratically, making it impossible to run on large graphs (like social networks or citation networks).

Modern Solutions to the Scaling Bottleneck:

- Sparse Attention Variants: Instead of computing the full $N \times N$ matrix, attention is restricted to specific subsets (e.g., only attending to the extended k-hop neighborhood or a randomly sampled set of distant nodes).
- Linear Attention / Performers: Uses mathematical kernel tricks to approximate the dense Softmax attention matrix without explicitly computing it, dropping the complexity to true linear $\mathcal{O}(N)$ time.
- Hybrid Local-Global Models: Uses cheap standard Message Passing for local structural edges, combined with a Graph Transformer that only routes global information through a small, manageable set of added "virtual nodes."

Graph Transformers: Injecting Topology

If a Graph Transformer computes attention between all nodes, it becomes entirely blind to the original graph structure! We must explicitly inject the topology back in via encodings.

1. Positional Encodings (Global Geometry)
   - Text Transformers use 1D sequence positions (1, 2, 3...), but graphs have no absolute order.
   - Laplacian PEs: We compute the eigenvectors of the Graph Laplacian ($\mathbf{L} = \mathbf{D} - \mathbf{A}$). They encode low-frequency structural signals; eigenvectors are not literal spatial coordinates, but capture smooth variations over the graph.
2. Structural Encodings (Local Topology)
   - Node-level: Injecting a node's *degree* or *eigenvector centrality* into its initial embedding so the model knows its local importance.
   - Pair-level (Spatial): Modifying the attention score between node i and j based on their *Shortest Path Distance* (SPD) in the original graph.

The Expressivity Problem

We have covered GCN, GraphSAGE, GAT, and Graph Transformers. Are these models powerful enough to distinguish any two different graphs?

No! Both paradigms have fundamental mathematical limits:

The Expressivity Problem

We have covered GCN, GraphSAGE, GAT, and Graph Transformers. Are these models powerful enough to distinguish any two different graphs?

No! Both paradigms have fundamental mathematical limits:

1. Standard Message Passing (GCN, GraphSAGE, GAT)
   - Upper-bounded by the **1-Weisfeiler-Lehman (1-WL) Test**. They can not distinguish certain symmetric structures (e.g., a hexagon vs. two triangles).
   - The Bottleneck: The choice of the **AGGREGATE** function matters deeply. (e.g., "Mean" and "Max" aggregators lose count information; only "Sum" reaches the 1-WL limit).
2. Graph Transformers
   - Global attention bypasses the local message-passing bottleneck, but introduces a new one: treating the graph as a featureless set.
   - The Bottleneck: The expressivity of a GT is entirely dictated by its Positional/Structural Encodings. Even advanced Laplacian encodings fail to distinguish strongly regular graphs due to eigenvalue symmetries.

Architecture 4: Graph Isomorphism Networks (GIN)

GIN (Xu et al., 2019) was designed specifically to maximize the mathematical expressive power of GNNs.

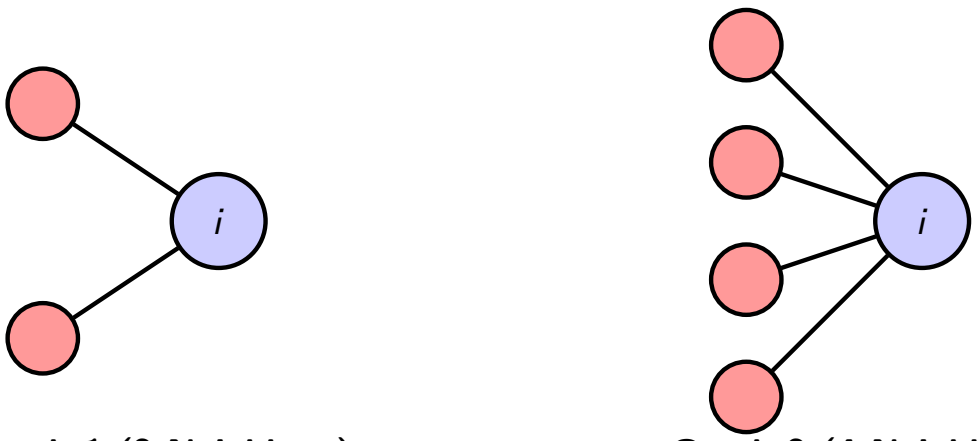
- It is proven to be as powerful as the 1-dim Weisfeiler-Lehman (WL) graph isomorphism test.
- It strictly uses SUM aggregation, and applies MLPs everywhere.

GIN Update Rule

$$h_i^{(t)} = \mathrm{MLP}^{(t)}\left((1+\epsilon^{(t)})h_i^{(t-1)} + \sum_{j\in\mathcal{N}(i)} h_j^{(t-1)}\right)$$

The learned parameter ϵ allows the network to distinguish the node's own features from its neighbors' features.

Why GIN uses SUM instead of MEAN/MAX



Graph 1 (2 Neighbors) **Graph 2 (4 Neighbors)**

- **Mean Aggregation:** Graph 1 $= \frac{Red+Red}{2} = Red$. Graph 2 $= \frac{4 \times Red}{4} = Red$. (Fails!)
- **Sum Aggregation:** Graph 1 $= 2 \times Red$. Graph 2 $= 4 \times Red$. (Succeeds!)
- GIN is crucial for Graph-Level tasks (like molecular chemistry) where subtle structural counts define properties.

The Limits of Deep GNNs: Two Major Bottlenecks

Why can't we simply stack 100 layers to learn long-range graph interactions? Deep GNNs suffer from two distinct mathematical constraints:

1. Over-smoothing (The Convergence Problem)
   - Mechanism: With every layer, nodes repeatedly average or aggregate features from a wider neighborhood.
   - Result: The representations of all nodes blur together, eventually converging to become mathematically indistinguishable.
   - Analogy: Mixing too many paint colors together until everything turns into a uniform muddy brown.
2. Over-squashing (The Bottleneck Problem)
   - Mechanism:In real graphs, the neighborhood size grows *exponentially* with each hop. Message passing forces this massive amount of data into a single, **fixed-size** vector.
   - Result: Important signals from distant nodes get "squashed" and lost as noise, preventing the model from learning long-range dependencies.
   - Analogy: Trying to compress a 1,000-page textbook into a single tweet. The nuanced, distant details are inevitably lost.

Architecture Summary Matrix

Architecture	Key Innovation	Best Use Case
GCN	Spectral approximation	Simple, efficient baseline. Good for Transductive node classification.
GraphSAGE	Neighbor Sampling	Scaling to massive graphs. Inductive learning on unseen nodes.
GAT	Edge Attention	Heterogeneous graphs where some neighbors matter more than others.
GIN	WL-Test Expressivity	Graph classification (e.g., drug discovery) requiring strict structure ID.

Practical Training 1: Deep GNNs & Regularization

If you must build a deeper GNN (e.g., 8+ layers), you need specific architectural tricks to prevent the network from collapsing via over-smoothing or over-fitting.

1. Residual (Skip) Connections

- The Trick: Add the input of the layer directly to its output: $\mathbf{H}^{(l+1)} = \text{GNN}(\mathbf{H}^{(l)}) + \mathbf{H}^{(l)}$.
- Why it works: It gives the model a "gradient highway" and ensures that a node retains its original, localized features even after many rounds of neighborhood aggregation. It directly combats over-smoothing.

2. DropEdge (Data Augmentation for Graphs)

- The Trick: Similar to standard Dropout, but instead of zeroing out node features, we randomly remove a percentage of *edges* from the adjacency matrix during each training epoch.
- Why it works: It acts as a powerful regularizer that prevents over-fitting. Crucially, it also slows down over-smoothing by artificially sparsifying the graph, limiting how fast information spreads.

Practical Training 2: Normalization in Graphs

Just like in CNNs or Transformers, aggregating hundreds of vectors can cause feature values to explode. Normalization stabilizes the training dynamics.

BatchNorm (Node-wise)

- Normalizes each feature channel across all nodes in the current training batch.
- Best For: Large, single-graph node classification tasks where the batch size (number of nodes) is sufficiently large and stable.

LayerNorm (Feature-wise)

- Normalizes across the feature dimension for each individual node, independently of the batch.
- Best For: Highly variable graph sizes (e.g., molecule datasets where one graph has 10 nodes and the next has 100) or Graph Transformers.

Normalization is typically applied *after* the aggregation step but *before* the non-linear activation function (e.g., ReLU).

Practical Training 3: Scalability & Mini-Batching

Standard deep learning loads data in isolated mini-batches. You can not do this easily with graphs because nodes are connected!

The "Neighborhood Explosion" Problem

- To compute the embedding for Node A in a 3-layer GNN, you need Node A's neighbors, *their* neighbors, and *their* neighbors. On dense graphs (like social networks), this quickly requires loading the entire graph into GPU memory.

Solutions for Large Graphs

- Neighbor Sampling (GraphSAGE): Instead of taking all neighbors, we randomly sample a fixed maximum number of neighbors (e.g., 10) per node at each layer. This caps the memory footprint.
- Subgraph Sampling (Cluster-GCN): We use a clustering algorithm to break the massive graph into smaller, densely connected subgraphs. We then train on these subgraphs as standard, isolated mini-batches.